



Advanced Web Programming

Ung Văn Giàu
Email: giau.ung@eiu.edu.vn



Working with Forms

Topic

We'll cover the following topics:

- Using controlled fields
- Using uncontrolled fields
- Using React Router Form
- Using native validation
- Using React Hook Form

Technical requirements

- Browser
- Node.js and npm
- Visual Studio Code



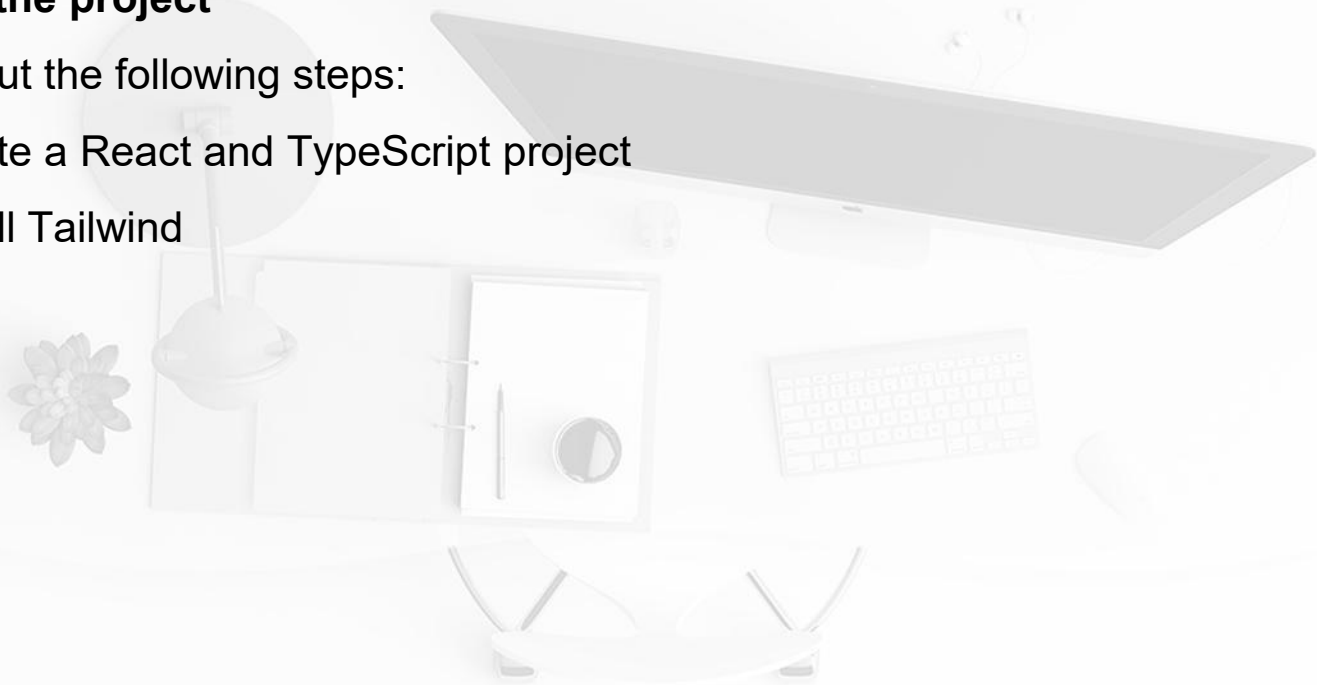
1. Using controlled fields

- We will build the first version of our contact form.
- It will contain fields for the user's **name**, **email address**, **contact reason**, and any **additional notes** the user may want to make.
- This method will involve the use of **controlled fields**, which is where field values are stored in the **state**.

1. Using controlled fields

Creating the project

- Carry out the following steps:
 - Create a React and TypeScript project
 - Install Tailwind



1. Using controlled fields

Creating a contact form

- Carry out the following steps:

- Create a file called **ContactPage.tsx** in the **src** folder

```
import { useState, FormEvent } from 'react';
```

- Add the following **type** alias under the import statement.

```
type Contact = {  
  name: string;  
  email: string;  
  reason: string;  
  notes: string;  
};
```

1. Using controlled fields

Creating a contact form

- Carry out the following steps:
 - Add the following **function** component:

```
export function ContactPage() {  
  return (  
    <div className="flex flex-col py-10 max-w-md  
      mx-auto">  
      <h2 className="text-3xl font-bold underline  
        mb-3">Contact Us</h2>  
      <p className="mb-3">  
        If you enter your details we'll get back to you  
        as soon as we  
        can.  
      </p>  
      <form></form>  
    </div>  
  );  
}
```


1. Using controlled fields

Creating a contact form

- Carry out the following steps:
 - Add the following fields inside the **form** element:

```
<form ...>
  <div>
    <label htmlFor="name">Your name</label>
    <input type="text" id="name" />
  </div>
  <div>
    <label htmlFor="email">Your email address</label>
    <input type="email" id="email" />
  </div>
  <div>
    <label htmlFor="reason">Reason you need to contact
      us</label>
    <select id="reason">
      <option value=""></option>
      <option value="Support">Support</option>
      <option value="Feedback">Feedback</option>
      <option value="Other">Other</option>
    </select>
  </div>
  <div>
    <label htmlFor="notes">Additional notes</label>
    <textarea id="notes" />
  </div>
</form>
```

1. Using controlled fields

Creating a contact form

- Carry out the following steps:
 - Add a **submit** button to the bottom of the **form** element

```
<form ...>
  ...
  <div>
    <button
      type="submit"
      className="mt-2 h-10 px-6 font-semibold bg-black
      text-white"
    >
      Submit
    </button>
  </div>
</form>
```

1. Using controlled fields

Creating a contact form

- Carry out the following steps:
 - The field containers will all have the same style, so create a variable for the style and assign it to all the field containers

```
const fieldStyle = "flex flex-col mb-2";
return (
  <div ...>
    ...
    <form ...>
      <div className={fieldStyle}>...</div>
      <div className={fieldStyle}>...</div>
      <div className={fieldStyle}>...</div>
      <div className={fieldStyle}>...</div>
      <div>
        <button
```

1. Using controlled fields

Creating a contact form

- Carry out the following steps:
 - Add the state to hold the field values

```
export function ContactPage() {  
  const [contact, setContact] = useState<Contact>({  
    name: "",  
    email: "",  
    reason: "",  
    notes: "",  
  });  
  const fieldStyle = ...;  
  ...  
}
```

1. Using controlled fields

Creating a contact form

- Carry out the following steps:
 - Bind the state to the **name** field editor

```
<div className={fieldStyle}>
  <label htmlFor="name">Your name</label>
  <input
    type="text"
    id="name"
    value={contact.name}
    onChange={(e) =>
      setContact({ ...contact, name: e.target.value })
    }
  />
</div>
```

1. Using controlled fields

Creating a contact form

- Carry out the following steps:
 - Repeat the same approach for the binding state to the other field editors

```
<input
  type="email"
  id="email"
  value={contact.email}
  onChange={(e) =>
    setContact({ ...contact, email: e.target.value })
  }
/>
```

```
<textarea
  id="notes"
  value={contact.notes}
  onChange={(e) =>
    setContact({ ...contact, notes: e.target.value })
  }
/>
```

```
<select
  id="reason"
  value={contact.reason}
  onChange={(e) =>
    setContact({ ...contact, reason: e.target.value })
  }
>
```

1. Using controlled fields

Creating a contact form

- Carry out the following steps:
 - Add a submit handler to the **form** element

```
function handleSubmit(e: FormEvent<HTMLFormElement>) {  
  e.preventDefault();  
  console.log('Submitted details:', contact);  
}  
const fieldStyle = ...;  
return (  
  <div>  
    ...  
    <form onSubmit={handleSubmit}>
```

1. Using controlled fields

Creating a contact form

- Carry out the following steps:
 - Open **App.tsx** and replace its content with the following

```
import { ContactPage } from './ContactPage';  
  
function App() {  
  return <ContactPage />;  
}  
  
export default App;
```


1. Using controlled fields

Creating a contact form

- We will now use the form and discover a potential performance problem.
- Carry out the following steps:
 - Run the app in development mode.
 - Open the browser **DevTools** and select the **Components** panel. Click on the **settings cog** to view the React DevTools settings and tick the **Highlight updates when components render** option.
 - Fill in the form and notice a blue green outline appears around the form every time a character is entered into a field.
 - Complete all the fields in the form and click on the **Submit** button.

1. Using controlled fields

Recap

- Controlling field values with the state can lead to performance problems.
- Having to bind the state to each field also feels a bit repetitive.



2. Using uncontrolled fields

- Uncontrolled fields are the opposite of controlled fields.
- Field values aren't controlled by state. Instead, native browser features are used to obtain field values.
- Carry out the following steps:
 - Open **ContactPage.tsx** and start by removing **useState** from the React import
 - At the top of the component function, remove the call to **useState**
 - Remove the value and **onChange** props from the field editors

2. Using uncontrolled fields

- Carry out the following steps:
 - Add a **name** attribute on all the field editors

```
<form onSubmit={handleSubmit}>
  <div className={fieldStyle}>
    <label htmlFor="name">Your name</label>
    <input type="text" id="name" name="name" />
  </div>
  <div className={fieldStyle}>
    <label htmlFor="email">Your email address</label>
    <input type="email" id="email" name="email" />
  </div>
  <div className={fieldStyle}>
    <label htmlFor="reason">
      Reason you need to contact us
    </label>
    <select id="reason" name="reason">
      ...
    </select>
  </div>
  <div className={fieldStyle}>
    <label htmlFor="notes">Additional notes</label>
    <textarea id="notes" name="notes" />
  </div>
  ...
</form>;
```

2. Using uncontrolled fields

- Carry out the following steps:
 - Add the following code in the submit handler to extract the field values before they are output to the console:

```
function handleSubmit(e: FormEvent<HTMLFormElement>) {  
  e.preventDefault();  
  const formData = new FormData(e.currentTarget);  
  const contact = {  
    name: formData.get('name'),  
    email: formData.get('email'),  
    reason: formData.get('reason'),  
    notes: formData.get('notes'),  
  } as Contact;  
  console.log('Submitted details:', contact);  
}
```

2. Using uncontrolled fields

Recap

- Uncontrolled fields don't have values stored in the state.
- Field values are obtained using **FormData**, which relies on field editors having a **name** attribute.
- **Notice** the reduced code in the implementation compared to the controlled fields implementation.

2. Using uncontrolled fields

- We will now try the form and check whether the form is re-rendered on every keystroke.
- Carry out the following steps:
 - In the running app, make sure **DevTools** is still open with the **Highlight updates when the components render** option still ticked.
 - Fill in the form and you will notice that the **re-render** outline **never appears**.
 - Complete all the fields in the form and click on the Submit button. The field values are output to the console just as they were before.

2. Using uncontrolled fields

Recap

- This implementation is shorter, more performant, and an excellent approach for simple forms.
- The key points are to include a **name** attribute for field editors and use the **FormData** interface to extract the form values.

3. Using React Router Form

- Carry out the following steps:
 - Install **React Router**: `npm i react-router-dom`
 - Create a file called **ThankYouPage.tsx** in the **src** folder

```
import { useParams } from 'react-router-dom';

export function ThankYouPage() {
  const { name } = useParams<{ name: string }>();
  return (
    <div className="flex flex-col py-10 max-w-md mx-auto">
      <div
        role="alert"
        className="bg-green-100 py-5 px-6 text-base text-green-700 "
      >
        Thanks {name}, we will be in touch shortly
      </div>
    </div>
  );
}
```

3. Using React Router Form

- Carry out the following steps:

- Open **App.tsx** and add the following imports from React Router

```
import {  
  createBrowserRouter,  
  RouterProvider,  
  Navigate  
} from 'react-router-dom';
```

- Add **contactPageAction** to the **import** statement for **ContactPage** and also import the **ThankYouPage** component:

```
import {  
  ContactPage,  
  contactPageAction  
} from './ContactPage';  
import { ThankYouPage } from './ThankYouPage';
```

3. Using React Router Form

- Carry out the following steps:
 - Still in **App.tsx**, set up routes that render the contact and thank you pages:

```
const router = createBrowserRouter([
  {
    path: '/',
    element: <Navigate to="contact" />,
  },
  {
    path: '/contact',
    element: <ContactPage />,
    action: contactPageAction,
  },
  {
    path: '/thank-you/:name',
    element: <ThankYouPage />,
  },
]);
```

3. Using React Router Form

- Carry out the following steps:
 - The last task in **App.tsx** is to change the App component to return **RouterProvider** with the route definitions:

```
function App() {  
  return <RouterProvider router={router} />;  
}
```

- Open **ContactPage.tsx** and add the following imports from React Router:

```
import {  
  Form,  
  ActionFunctionArgs,  
  redirect,  
} from 'react-router-dom';
```

3. Using React Router Form

- Carry out the following steps:
 - In the JSX, change the form element to be a React Router **Form** component and remove the onSubmit attribute:

```
<Form method="post">  
  ...  
</Form>
```

3. Using React Router Form

- Carry out the following steps:
 - Move the **handleSubmit** function outside the component, to the bottom of the file. Rename the function to **contactPageAction**, allow it to be exported, and make it asynchronous:

```
export async function contactPageAction(  
  e: FormEvent<HTMLFormElement>  
) {  
  e.preventDefault();  
  const formData = new FormData(e.currentTarget);  
  const contact = {  
    name: formData.get('name'),  
    email: formData.get('email'),  
    reason: formData.get('reason'),  
    notes: formData.get('notes'),  
  } as Contact;  
  console.log('Submitted details:', contact);  
}
```

3. Using React Router Form

- Carry out the following steps:

- Change the parameters on **contactPageAction** to the following:

```
export async function contactPageAction({  
  request,  
}: ActionFunctionArgs)
```

- Remove the **e.preventDefault()** statement in **contactPageAction**
- Change the **formData** assignment to get the data from the React Router's **request** object: `const formData = await request.formData();`

3. Using React Router Form

- Carry out the following steps:
 - The last change to the **contactPageAction** function is to redirect to the thank you page at the end of the submission:

```
export async function contactPageAction({
  request,
}: ActionFunctionArgs) {
  ...
  return redirect(
    `/${thank-you}/${formData.get('name')}`
  );
}
```


3. Using React Router Form

- Carry out the following steps:
 - Remove the **FormEvent** import
 - Run the app in development mode.
 - The app will automatically redirect to the **Contact** page. Complete the form and submit it. The app will redirect to the thank you page:

3. Using React Router Form

Recap

- React Router's **Form** component is a wrapper around the HTML **form** element
- The form is submitted to the current route by default, but can be submitted to a different path using the **path** attribute
- We can write logic inside the submission process using an **action** function defined on the route that is submitted to

4. Using native validation

▪ Description:

- We will add the required validation to the name, email, and reason fields and ensure that the email matches a particular pattern.
- We will use standard HTML form validation to implement these rules.

4. Using native validation

- Carry out the following steps:
 - In **ContactPage.tsx**, add a **required** attribute to the name, email, and reason field editors to add HTML form required validation for these fields:

```
<Form method="post">
  <div className={fieldStyle}>
    ...
    <input type="text" id="name" name="name" required />
  </div>
  <div className={fieldStyle}>
    ...
    <input type="email" id="email" name="email" required
  />
</div>
  <div className={fieldStyle}>
    ...
    <select id="reason" name="reason" required >...</
      select>
    </div>
    ...
</Form>
```

4. Using native validation

- Carry out the following steps:
 - Add the following pattern-matching validation on the **email** field editor:

```
<input
  type="email"
  id="email"
  name="email"
  required
  pattern="\S+@\S+\.\S+"
/>
```

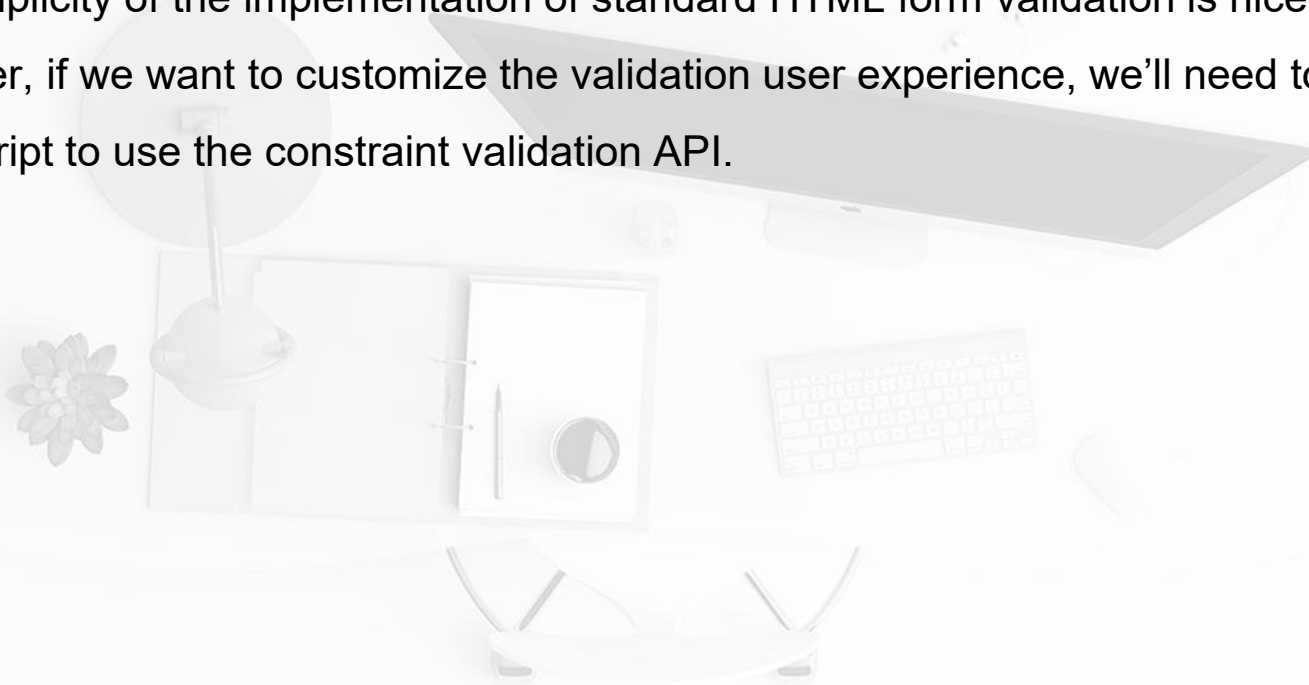
4. Using native validation

- Carry out the following steps:
 - In the running app, without populating any fields, submit the form and give your opinion.
 - Correctly fill in the email field so that it is valid. Then, move on to experiment with the reason field validation. Try to select the blank reason and you will find that a validation error occurs.
 - Correctly fill in all the fields and submit the form. You will find that the thank you message appears as it did previously.

4. Using native validation

Recap

- The simplicity of the implementation of standard HTML form validation is nice.
- However, if we want to customize the validation user experience, we'll need to write JavaScript to use the constraint validation API.



5. Using React Hook Form

Understanding React Hook Form

- React Hook Form is a React library for building forms.
- It is very flexible and can be used for simple forms such as our contact form, as well as large forms with complex validation and submission logic.
- It is also very performant and optimised not to cause unnecessary re-renders.

5. Using React Hook Form

Understanding React Hook Form

- The key part of React Hooks Form is a **useForm** hook, which returns useful functions and the state.
- The following code snippet shows the useForm hook being called:

```
const {
  register,
  handleSubmit,
  formState: { errors, isSubmitting, isSubmitSuccessful }
} = useForm<FormData>();
```

- **useForm** has a generic type parameter for the type of the field values.

5. Using React Hook Form

Understanding the register function

- A key function that **useForm** returns is a **register** function, which takes in a unique field name and returns the following in an **object** structure:
 - An **onChange** handler, which happens when the field editor's value changes
 - An **onBlur** handler, which happens when the field editor loses focus
 - A reference to the field editor element
 - The field name

5. Using React Hook Form

Understanding the register function

- These items returned from the **register** function are spread onto the field editor element to allow React Hook Form to efficiently track its value.
- The following code snippet allows a name field editor to be tracked by React Hook Form:

```
<input {...register('name')} />
```

5. Using React Hook Form

Understanding the register function

- After the result of **register** has been spread on to the input element, it will contain **ref**, **name**, **onChange**, and **onBlur** attributes.

```
<input  
  ref={someVariableInRHF}  
  name="name"  
  onChange={someHandlerInRHF}  
  onBlur={anotherHandlerInRHF}  
/>
```

5. Using React Hook Form

Specifying validation

- Field validation is defined in the **register** field in an options parameter:

```
<input {...register('name', {required: true})} />
```

- The associated error message can be defined as an alternative to the **true** flag:

```
<input  
  {...register('name', { required: 'You must enter a name' })}  
/>
```

- There are a range of different validation rules that can be applied. (<https://react-hook-form.com/get-started#applyvalidation>)

5. Using React Hook Form

Obtaining validation errors

- The **useForm** returns a state called **errors**, which contains the form validation errors. The **errors** state is an **object** containing invalid field error messages.
- For example, if a **name** field is invalid because a **required** rule has been violated, the **errors** object could be as follows:

```
{
  name: {
    message: 'You must enter your email address',
    type: 'required'
  }
}
```

5. Using React Hook Form

Obtaining validation errors

- Fields in a valid state don't exist in the **errors** object, so a field validation error message can be conditionally rendered.

```
{errors.name && <div>{errors.name.message}</div>}
```

5. Using React Hook Form

Handling submission

- The **useForm** hook also returns a handler called **handleSubmit** that can be used for form submission.
- **handleSubmit** takes in a function that React Hook Form calls when it has successfully validated the form.
- Here's an example of **handleSubmit** being used:

```
function onSubmit(data: FormData) {  
  console.log('Submitted data:', data);  
}  
return (  
  <form onSubmit={handleSubmit(onSubmit)}>  
    </form>  
);
```


5. Using React Hook Form

Handling submission

- The **isSubmitting** state can be used to disable elements whilst the form is being submitted.
- The following example disables the **submit** button while the form is being submitted:

```
<button type="submit" disabled={isSubmitting}>Submit</button>
```

- **isSubmitSuccessful** can be used to conditionally render a successful submission message:

```
if (isSubmitSuccessful) {  
  return <div>The form was successfully submitted</div>;  
}
```

5. Using React Hook Form

Using React Hook Form

- Carry out the following steps:
 - Install React Hook Form: **npm i react-hook-form**
 - Open **ContactPage.tsx** and add an **import** statement for **useForm** from React Hook Form:

```
import { useForm } from 'react-hook-form';
```
 - Remove **Form**, **redirect**, and **ActionFunctionArgs** from the React Router **import** statement and replace them with **useNavigate**:

```
import { useNavigate } from 'react-router-dom';
```

5. Using React Hook Form

Using React Hook Form

- Carry out the following steps:
 - Add the following call to **useForm** at the top of the **ContactPage** component and destructure the **register** and **handleSubmit** functions:

```
export function ContactPage() {  
  const { register, handleSubmit } = useForm<Contact>();  
  ...  
}
```

5. Using React Hook Form

Using React Hook Form

- Carry out the following steps:
 - Add the following call to **useNavigate** after the call to **useForm** to get a function that we can use to perform navigation:

```
export function ContactPage() {  
  const { register, handleSubmit } = useForm<Contact>();  
  const navigate = useNavigate();  
}
```

5. Using React Hook Form

Using React Hook Form

- Carry out the following steps:
 - In the JSX, replace the use of React Router's **Form** element with a native **form** element. Remove the **method** attribute on the **form** element, and replace it with the following **onSubmit** attribute:

```
<form onSubmit={handleSubmit(onSubmit)}>  
  ...  
</form>
```

5. Using React Hook Form

Using React Hook Form

- Carry out the following steps:
 - Add the following **onSubmit** function after the call to **useNavigate**. React Hook Form will call this function with the form data after it has ensured the form is valid:

```
const navigate = useNavigate();  
function onSubmit(contact: Contact) {  
  console.log('Submitted details:', contact);  
  navigate(`/thank-you/${contact.name}`);  
}
```

- Remove the **contactPageAction** function at the bottom of the file

5. Using React Hook Form

Using React Hook Form

- Carry out the following steps:
 - Replace the **name** attribute on the field editors with a call to **register**. Pass the field name into **register** and spread the result of **register**:

```
<form onSubmit={handleSubmit(onSubmit)}>
  <div className={fieldStyle}>
    <label htmlFor="name">Your name</label>
    <input ... {...register('name')} />
  </div>
  <div className={fieldStyle}>
    <label htmlFor="email">Your email address</label>
    <input ... {...register('email')} />
  </div>
  <div className={fieldStyle}>
    <label htmlFor="reason">Reason you need to contact
      us</label>
    <select ... {...register('reason')}>
      ...
    </select>
  </div>
  <div className={fieldStyle}>
    <label htmlFor="notes">Additional notes</label>
    <textarea ... {...register('notes')} />
  </div>
  ...
</Form>
```

5. Using React Hook Form

Using React Hook Form

- Carry out the following steps:
 - Open **App.tsx** and remove **contactPageAction** from the **import** statement for **ContactPage** and remove it from the **/contact** route:

```
import { ContactPage } from './ContactPage';
...
const router = createBrowserRouter([
  {
    path: '/',
    element: <Navigate to="contact" />,
  },
  {
    path: '/contact',
    element: <ContactPage />
  },
  {
    path: '/thank-you/:name',
    element: <ThankYouPage />,
  }
]);
```


5. Using React Hook Form

Using React Hook Form

- Carry out the following steps:
 - Run the app.
 - Open the **React DevTools** and ensure **the Highlight updates when components render** option is still ticked so that we can observe when the form is re-rendered.
 - Fill in the form with valid values. The re-render outline doesn't appear when the field values are entered because they aren't using the state to cause a re-render yet.
 - Click on the **Submit** button. After the form has been successfully submitted, the field values are output to the console and the thank you page appears.

5. Using React Hook Form

Adding validation

- Carry out the following steps:
 - Open **ContactPage.tsx** and add the **FieldError** type to the React Hook Form **import** statement:

```
import { useForm, FieldError } from 'react-hook-form';
```

- Destructure the **errors** state from **useForm**:

```
const {  
  register,  
  handleSubmit,  
  formState: { errors }  
} = useForm<Contact>();
```

5. Using React Hook Form

Adding validation

- Carry out the following steps:
 - Add a **noValidate** attribute to the **form** element to prevent any native HTML validation:

```
<form noValidate onSubmit={handleSubmit(onSubmit)}>
```
 - Remove the native HTML validation rules on all the field editors by removing the validation **required** and **pattern** attributes.

5. Using React Hook Form

Adding validation

- Carry out the following steps:
 - Add the required validation rules to the **register** function for the name, email, and reason fields

```
<div className={fieldStyle}>
  <label htmlFor="name">Your name</label>
  <input
    type="text"
    id="name"
    {...register('name', {
      required: 'You must enter your name'
    })}
  />
</div>
<div className={fieldStyle}>
  <label htmlFor="email">Your email address</label>
  <input
    type="email"
    id="email"
    {...register('email', {
      required: 'You must enter your email address'
    })}
  />
</div>
<div className={fieldStyle}>
  <label htmlFor="reason">Reason you need to contact us</label>
  <select
    id="reason"
    {...register('reason', {
      required: 'You must enter the reason for contacting us'
    })}
  >
    ...
  </select>
</div>
```

5. Using React Hook Form

Adding validation

- Carry out the following steps:
 - Add an additional rule for the email field to ensure it matches a particular pattern:

```
<input
  type="email"
  id="email"
  {...register('email', {
    required: 'You must enter your email address',
    pattern: {
      value: /\S+@\S+\.\S+/,
      message: 'Entered value does not match email
        format',
    }
  })}
/>
```

5. Using React Hook Form

Adding validation

- Carry out the following steps:
 - We will style fields if they are invalid. Each field is going to have the same styling logic, so define the style in a function.

```
function getEditorStyle(fieldError: FieldError |  
undefined) {  
  return fieldError ? 'border-red-500' : '';  
}  
  
return (  
  <div>  
    ...  
  </div>  
) ;
```

5. Using React Hook Form

Adding validation

- Carry out the following steps:
 - Use the **getEditorStyle** function to style the name, email, and reasons fields when invalid:

```
<div className={fieldStyle}>
  <label htmlFor="name">Your name</label>
  <input ... className={getEditorStyle(errors.name)} />
</div>

<div className={fieldStyle}>
  <label htmlFor="email">Your email address</label>
  <input ... className={getEditorStyle(errors.email)} />
</div>

<div className={fieldStyle}>
  <label htmlFor="reason">Reason you need to contact us</
label>
  <select ... className={getEditorStyle(errors.reason)} >
    ...
  </select>
</div>
```

5. Using React Hook Form

Adding validation

- Carry out the following steps:
 - Let's display the validation error under each field when it's invalid. The structure and style of the error will be the same for each field with a varying message, so we will create a reusable validation error component.

5. Using React Hook Form

Adding validation

- Carry out the following steps:
 - Create a file in the **src** folder called **ValidationError.tsx** with the following content:

```
import { FieldError } from 'react-hook-form';

type Props = {
  fieldError: FieldError | undefined;
};

export function ValidationError({ fieldError }: Props) {
  if (!fieldError) {
    return null;
  }

  return (
    <div role="alert" className="text-red-500 text-xs mt-1">
      {fieldError.message}
    </div>
  );
}
```

5. Using React Hook Form

Adding validation

- Carry out the following steps:
 - Return to **ContactPage.tsx** and import the **ValidationError** component:

```
import { ValidationError } from './ValidationError';
```

5. Using React Hook Form

Adding validation

- Carry out the following steps:
 - Add instances of **ValidationError** beneath each field editor:

```
<div className={fieldStyle}>
  <label htmlFor="name">Your name</label>
  <input ... />
  <ValidationError fieldError={errors.name} />
</div>
<div className={fieldStyle}>
  <label htmlFor="email">Your email address</label>
  <input ... />
  <ValidationError fieldError={errors.email} />
</div>
<div className={fieldStyle}>
  <label htmlFor="reason">Reason you need to contact us</label>
  <select ... >...</select>
  <ValidationError fieldError={errors.reason} />
</div>
```

5. Using React Hook Form

Adding validation

- That completes the implementation of the form validation. We will test our enhanced form in the following steps:
 - In the running app, ensure the **Highlight updates when components render** option is still ticked in **DevTools** so we can observe when the form is re-rendered.
 - Click on the **Submit** button without filling in the form.
 - Fill in the form and click on the **Submit** button. The validation errors appear. You will also notice that the form now re-renders when submitted because of the errors state.
 - Fill in the form correctly and submit it. The field values are output to the console and the thank you page appears.

5. Using React Hook Form

Adding validation

- Validation actually happens when the form is submitted. We will change the **validation** to happen every time a field editor **loses focus**.
- Carry out the following steps:
 - On **ContactPage.tsx**, add the following argument to the **useForm** call:

```
const {
  register,
  handleSubmit,
  formState: { errors },
} = useForm<Contact>({
  mode: "onBlur",
  reValidateMode: "onBlur"
});
```

5. Using React Hook Form

Recap

- React Hook Form doesn't cause unnecessary re-renders when a form is filled in.
- React Hook Form's **register** function needs to be spread on field editors. This function allows field values to be efficiently tracked and allows validation rules to be specified.
- React Hook Form's submit handler automatically prevents a server post and ensures the form is valid before our submission logic is called.

Summary

- **Field values** in a form can be controlled by the state. However, this leads to lots of unnecessary re-rendering of the form.
- **Not controlling field values** with the state and using the **FormData** interface to retrieve field values instead is more performant and requires less code.
- **React Router's Form** component, which is a wrapper around the native form element. It submits data to a client-side route instead of a server. However, it doesn't cover validation.
- **React Hook Form** provides a better validation user experience. It contains a **useForm** hook that returns useful functions and a state. The library doesn't cause unnecessary renders, so it is very performant.



Q&A